

SIH26122 - Extractor Training & Build Pipeline

AI/Core implementation specification for the extraction layer

1. PROJECT CONTEXT

SIH26122 is an Oil India Limited Software / Smart Automation problem about bridging structured infrastructure-project schedules and messy field-execution information. Plans contain executable L5/L6 activities, while actual progress may arrive through daily reports, site diaries, spreadsheets and supervisor updates. The system must extract execution facts so a later layer can link them to the schedule. [SIH26122 problem statement]

Current phase: build only the extractor. Do not implement schedule matching, candidate ranking, RAG, OCR, ASR, dashboards or schedule updates yet.

Extractor job: Raw report -> relevance decision -> one or more structured execution events. Missing information becomes null; uncertainty becomes needs_review=true; unsupported facts are never invented.

2. FROZEN OUTPUT SCHEMA

Use this as the canonical v1 output contract. The extractor must not emit a schedule_activity_id.

```
{ "relevance": { "is_relevant": true, "confidence": 0.98 }, "events": [ { "event_id": "EVT-001", "activity": { "description": "24 inch spool erection", "action": "erection", "execution": { "status": "completed", "assertion": "affirmed", "progress_percent": 100 }, "time": { "start": "2026-09-01T10:00:00", "end": "2026-09-01T16:00:00", "certainty": "explicit" }, "context": { "discipline": "Piping", "location": "Rack B", "line_number": "24", "equipment": null }, "quantity": { "completed": null, "total": null, "unit": null }, "issue": { "type": null, "reason": null }, "evidence": { "source_text": "Piping crew completed erection of 24 inch spool at Rack B from 10 AM to 4 PM." }, "confidence": { "overall": 0.95, "activity": 0.97, "status": 0.99, "time": 0.93 }, "needs_review": false, "warnings": [ ] } ] }
```

Rules: multiple activities = multiple events; missing values = null; preserve evidence; never fabricate; negation and uncertainty remain explicit.

3. RECOMMENDED ARCHITECTURE

RAW REPORT | v Input normalization | v Relevance + event extraction | v LLM structured output | v Pydantic validation | v Deterministic consistency checks |---- safe repair/retry |---- needs_review when not safely resolvable v Validated extraction JSON | v Evaluation / storage

Use an LLM for semantic interpretation, but surround it with deterministic controls. Valid JSON does not imply correct facts.

4. AI MODEL CHOICE

First model to benchmark: Qwen3-8B. Its current model card describes 8.2B parameters, 100+ languages/dialects, hybrid thinking/non-thinking modes, and 32K native context. This makes it a strong first candidate for English/Hindi/Hinglish site language and local experimentation. [Qwen3 model card]

Quality challenger: gpt-oss-20b. OpenAI describes gpt-oss-20b as a 21B-total / 3.6B-active open-weight model with structured-output and fine-tuning support. Use it as a comparison if local resources allow. [OpenAI gpt-oss-20b]

Do not choose the winner by reputation. Benchmark both on the same frozen extractor test set using field accuracy, status/time accuracy, relevance F1, unsafe-extraction rate and latency.

5. TECH STACK

Layer	Recommendation	Role
Language model	Qwen3-8B	Primary extractor benchmark
Comparison model	gpt-oss-20b	Optional quality comparison
Model server	Ollama first	Fast local development
Structured output	JSON Schema	Constrain response shape
Validation	Pydantic v2	Schema/type validation
Runtime	Python 3.11+	Pipeline and evaluation
Data	JSONL + CSV	Reports and labels
Testing	pytest	Regression/invariants
Later serving	vLLM	Higher throughput if required

Ollama supports JSON-schema constrained structured outputs and recommends validating results with Pydantic; lower temperature can improve deterministic extraction. [Ollama structured outputs]

vLLM supports JSON-schema constrained structured outputs and is a later option for higher-throughput serving. [vLLM structured outputs]

6. SYSTEM PROMPT DESIGN

You are a construction project progress extraction engine. Read only the supplied field report. Return only the defined JSON schema. Rules: - Extract only explicitly stated or clearly implied facts. - Never invent dates, quantities, locations, identifiers or activities. - Missing information = null. - One activity = one event; multiple activities = multiple events. - Negated information must remain negated. - Uncertain information must remain uncertain and normally require review. - Irrelevant/non-project text -> is_relevant=false and events=[]. - Understand English, Hindi, Hinglish, shorthand and broken site language. - Preserve supporting evidence. - Do not output schedule IDs or matching decisions.

Dynamic user input should contain the report text and only the metadata actually available to the extractor (for example report_date and source discipline). Do not leak schedule facts into the extraction prompt.

7. DATA REQUIRED FOR TRAINING/DEVELOPMENT

Use the existing ~1,000-report corpus as the starting point. It should cover normal, shorthand, paraphrased, noisy, multi-event, partial, missing-field, relative/overnight, negative, uncertain, delay/suspended, conflict and irrelevant reports.

Most important: diversity must be structural, not just synonym replacement. Vary word order, sentence count, shorthand, context placement, explicitness and conversational style.

8. TRAINING PIPELINE

Stage	Action
1. Prompt baseline	Run Qwen3-8B with structured output; no fine-tuning.
2. Evaluate	Run the frozen test set and collect field-level failures.
3. Diagnose	Group failures by relevance, events, status, progress, time, context, quantity, negation and uncertainty.
4. Improve	Fix prompt, validators and genuinely missing data coverage.
5. Re-evaluate	Use validation data; keep final test untouched.
6. Optional SFT	Only if prompt-only performance is insufficient.
7. LoRA/QLoRA	Train on development data only.
8. Final test	Evaluate once on the frozen test set.

Do not fine-tune first. With a synthetic dataset of around 1,000 examples, structured prompting plus validation may already be strong. Fine-tuning can overfit the generator's style and must be justified by measured errors.

9. FINE-TUNING PLAN (ONLY IF NEEDED)

Item	Recommendation
Base model	Qwen3-8B first
Method	SFT with LoRA/QLoRA
Input	Raw report + allowed metadata
Target	Exact extractor JSON
Training	Development split only
Validation	Held-out validation split
Final test	Frozen; never tune against it
Inference	Structured JSON output
Post-processing	Pydantic + deterministic rules

Use fine-tuning only if it improves held-out generalization, not merely training loss or the development score.

10. VALIDATION RULES

Check	Required behavior
Progress	0-100 only
Quantity	Non-negative; completed <= total when comparable
Time	Parseable timestamps when present
Overnight	Allow date rollover when evidence supports it
Missing data	null, never fabricated

Check	Required behavior
Relevance	false -> zero events
Evidence	Every event contains supporting source text
Negation	Negated claims cannot become completion
Uncertainty	Flag needs_review when appropriate
Contradiction	Conflicting status/progress -> unknown + review

11. TRAIN / VALIDATION / TEST

Split	Target	Use
Development	70%	Prompt tuning / fine-tuning
Validation	15%	Model selection
Frozen test	15%	Final evaluation only

Prevent leakage at the report-family / underlying-activity level. Near-identical variants of one generated event must not be split across development and final test.

The hardest test set should contain unseen combinations of wording, context and report structure so success reflects semantic understanding rather than memorization.

12. EVALUATION

Metric	Purpose
Relevance precision/recall/F1	Reject unrelated inputs correctly
Event precision/recall/F1	Find correct number of events
Field accuracy	Activity/action/status/progress/time/context/quantity
Datetime accuracy	Correct start/end interpretation
Evidence coverage	Claims have source support
Needs-review accuracy	Uncertain cases are flagged
Unsafe extraction rate	Confident incorrect facts instead of null/review

Report results by case type: normal, shorthand, paraphrase, noisy, multi-event, partial, negative, uncertain, delay/suspended, conflict and irrelevant.

Primary reliability metric: unsafe extraction rate. A valid JSON response can still be wrong; wrong facts are more dangerous than a cautious null/review.

13. ERROR-DRIVEN DEVELOPMENT

```
MODEL OUTPUT | +--> schema failure -----> repair/retry | +--> wrong event -----> model/data error | +--> wrong field
-----> field-specific analysis | +--> unsupported guess ---> strengthen null/abstain rule | +--> ambiguity ----->
needs_review | v Re-run validation/test
```

For each failure, decide whether the problem is the report, gold label, prompt/model, validator, or genuine ambiguity. Do not change gold labels merely to improve the score.

14. IMPLEMENTATION FILE PLAN

File	Purpose
src/schema.py	Pydantic extraction models + JSON schema
src/prompt.py	System prompt
src/extractor_llm.py	LLM call + structured output
src/validators.py	Semantic consistency checks
src/extractor.py	Stable public extractor API
src/eval_extractor.py	Metrics + errors
src/error_analysis.py	Failure categorization
src/train_lora.py	Optional later fine-tuning
src/test_extractor.py	Unit/invariant tests

15. MINIMUM BUILD ORDER

Step	Deliverable
1	Freeze schema
2	Implement Pydantic validation
3	Connect Qwen3-8B through Ollama structured output
4	Run a small sample and inspect failures
5	Run full extractor evaluation
6	Improve prompt + validators + data
7	Benchmark gpt-oss-20b if useful
8	Consider LoRA/QLoRA only if justified
9	Freeze final test result

16. WHAT NOT TO BUILD NOW

Do not build schedule matching, candidate retrieval, RAG, OCR, ASR, PMIS integration, frontend or schedule updates in this phase. Do not use gold labels at inference time. Do not tune against the final test set.

17. CURRENT DATA STATUS

The repository now provides roughly 1,000 reports with explicit case categories and a controlled regression benchmark. The latest evaluation is non-perfect: relevance F1 is about 0.985, event F1 about 0.973, overall field accuracy about 0.972, status accuracy about 0.905, progress accuracy about 0.922 and end-time accuracy about 0.915. Treat these as development/regression numbers, not real-world accuracy.

18. FINAL DIRECTIVE FOR CLAUDE CODE

Implement a conservative structured extractor first. Use Qwen3-8B + Ollama structured output + Pydantic validation + deterministic consistency checks. Establish a prompt-only baseline, perform error analysis, improve prompt/data, and only then consider LoRA/QLoRA. Preserve the frozen test set and report field-level and unsafe-extraction metrics.

19. RESEARCH BASIS

Qwen3-8B model card: 8.2B parameters, 100+ languages/dialects, hybrid reasoning modes and 32K native context. [Qwen3 model card]

OpenAI gpt-oss-20b model page: 21B total / 3.6B active parameters, structured outputs and fine-tuning support. [OpenAI gpt-oss-20b]

Ollama structured outputs documentation: JSON-schema constrained generation and Pydantic validation. [Ollama structured outputs]

vLLM structured outputs documentation: JSON-schema constrained generation. [vLLM structured outputs]